

```
from google.colab import files
uploaded = files.upload()
```

Choose Files

Housing.csv

Housing.csv(text/csv) - 29981 bytes, last modified: 3/3/2026 - 100% done

Saving Housing.csv to Housing.csv

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

```
df = pd.read_csv('/content/Housing.csv')
df.head()
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	pr
0	13300000	7420	4	2	3	yes	no	no	no	yes	2	
1	12250000	8960	4	4	4	yes	no	no	no	yes	3	
2	12250000	9960	3	2	2	yes	no	yes	no	no	2	
3	12215000	7500	4	2	2	yes	no	yes	no	yes	3	
4	11410000	7420	4	1	2	yes	yes	yes	no	yes	2	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df.info()
df.describe()
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 545 entries, 0 to 544
Data columns (total 13 columns):
Column Non-Null Count Dtype
--- ---
0 price 545 non-null int64
1 area 545 non-null int64
2 bedrooms 545 non-null int64
3 bathrooms 545 non-null int64
4 stories 545 non-null int64
5 mainroad 545 non-null object
6 guestroom 545 non-null object
7 basement 545 non-null object
8 hotwaterheating 545 non-null object
9 airconditioning 545 non-null object
10 parking 545 non-null int64
11 prefarea 545 non-null object
12 furnishingstatus 545 non-null object
dtypes: int64(6), object(7)
memory usage: 55.5+ KB

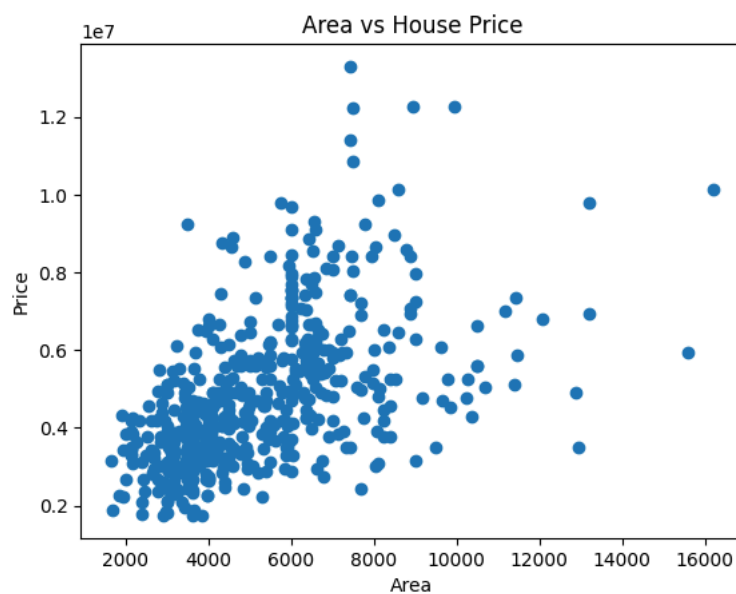
	price	area	bedrooms	bathrooms	stories	parking
count	5.450000e+02	545.000000	545.000000	545.000000	545.000000	545.000000
mean	4.766729e+06	5150.541284	2.965138	1.286239	1.805505	0.693578
std	1.870440e+06	2170.141023	0.738064	0.502470	0.867492	0.861586
min	1.750000e+06	1650.000000	1.000000	1.000000	1.000000	0.000000
25%	3.430000e+06	3600.000000	2.000000	1.000000	1.000000	0.000000
50%	4.340000e+06	4600.000000	3.000000	1.000000	2.000000	0.000000
75%	5.740000e+06	6360.000000	3.000000	2.000000	2.000000	1.000000
max	1.330000e+07	16200.000000	6.000000	4.000000	4.000000	3.000000

```
# Check missing values
df.isnull().sum()
```

	0
price	0
area	0
bedrooms	0
bathrooms	0
stories	0
mainroad	0
guestroom	0
basement	0
hotwaterheating	0
airconditioning	0
parking	0
prefarea	0
furnishingstatus	0

dtype: int64

```
# Scatter plot: Area vs Price
plt.scatter(df['area'], df['price'])
plt.xlabel("Area")
plt.ylabel("Price")
plt.title("Area vs House Price")
plt.show()
```



```
X = df[['area']]      # Independent variable
y = df['price']        # Dependent variable
```

```
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

LinearRegression ⓘ ?

```
LinearRegression()
```

```
# Prediction
y_pred = model.predict(X_test)
```

```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)

print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
```

```
Mean Absolute Error (MAE): 1474748.1337969352
Mean Squared Error (MSE): 3675286604768.185
```

```
X_multi = df[['area', 'bedrooms', 'bathrooms']]
y = df['price']
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error

X_train, X_test, y_train, y_test = train_test_split(
    X_multi, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
```

```
MAE: 1265275.6699454375
MSE: 2750040479309.0522
```

```
coeff_df = pd.DataFrame(
    model.coef_,
    X_multi.columns,
    columns=['Coefficient']
)
coeff_df
```

	Coefficient	
area	3.454666e+02	
bedrooms	3.601977e+05	
bathrooms	1.422320e+06	

Next steps:

[Generate code with coeff_df](#)[New interactive sheet](#)Start coding or [generate](#) with AI.